

HERMES AGENT

\$ a persistent, tool-wielding, self-improving AI agent –
not another chat window.█

● THE THESIS

A purpose-built agent architecture — memory, tools, skills, scheduling, and safety rails — is **meaningfully superior** to using GPT or Claude in isolation.

● WHAT YOU'LL SEE

Problem → Solution → Architecture → Safety & Human-in-the-Loop → Capabilities → Real-world impact. Every claim grounded in a running system.

● BUILD YOUR OWN

Docs & user stories:
hermes-agent.nousresearch.com/docs/user-stories

The Problem: Chat Windows Are Stateless, Blind & Passive

Standalone LLM interfaces (ChatGPT, Claude.ai, Gemini) are brilliant reasoners trapped in a box. Five structural limits break real workflows:

● NO PERSISTENCE

You type → it answers → the session ends → **it forgets everything**. Every conversation restarts from zero.

● NO SYSTEM ACCESS

No terminal, no files, no browser, no APIs. It can *describe* the fix — it can't **apply** it.

● NO AUTONOMY

Nothing runs while you're away. No scheduled jobs, no background execution, no "check this every Sunday."

● NO LEARNING

Same experience every time. Workflows you painstakingly explain today must be re-explained tomorrow.

● SINGLE SURFACE

One web tab. Not in your terminal, not on your phone, not in your team chat, not in your IDE.

● WHY IT MATTERS

Students & developers need agents that are **reliable, controllable, and continuous** — not one-shot answers.

The real cost is compounding: copy-paste glue work, context re-explained daily, tasks that die when the tab closes, and zero accumulated capability. The model gets smarter every year — **your workflow doesn't**. Agentic engineering is the discipline of fixing that gap responsibly.

The Solution: A Swappable-Brain Agent With a Body

Hermes keeps the LLM as the reasoning core — currently **Claude Sonnet 4 (Anthropic)** — and wraps it in everything a chat window lacks. The brain is swappable across 20+ providers (Anthropic, OpenAI, OpenRouter, Google, DeepSeek, xAI, local models) mid-workflow; nothing else changes.

● CHAT WINDOW VS HERMES

Standalone LLM (GPT / Claude.ai)

- └ type → answer → **session ends** → **forgets**
- └ **no** files · terminal · browser · APIs
- └ **no** actions, no automation
- └ **no** learning — same every time

Hermes Agent (this system)

- └ **persistent memory** across ALL sessions
- └ **full system access**: terminal, files, web, APIs
- └ **learns** workflows → saves skills → improves
- └ **runs autonomously**: cron jobs, background agents
- └ **meets you anywhere**: Telegram, CLI, IDE, Discord...

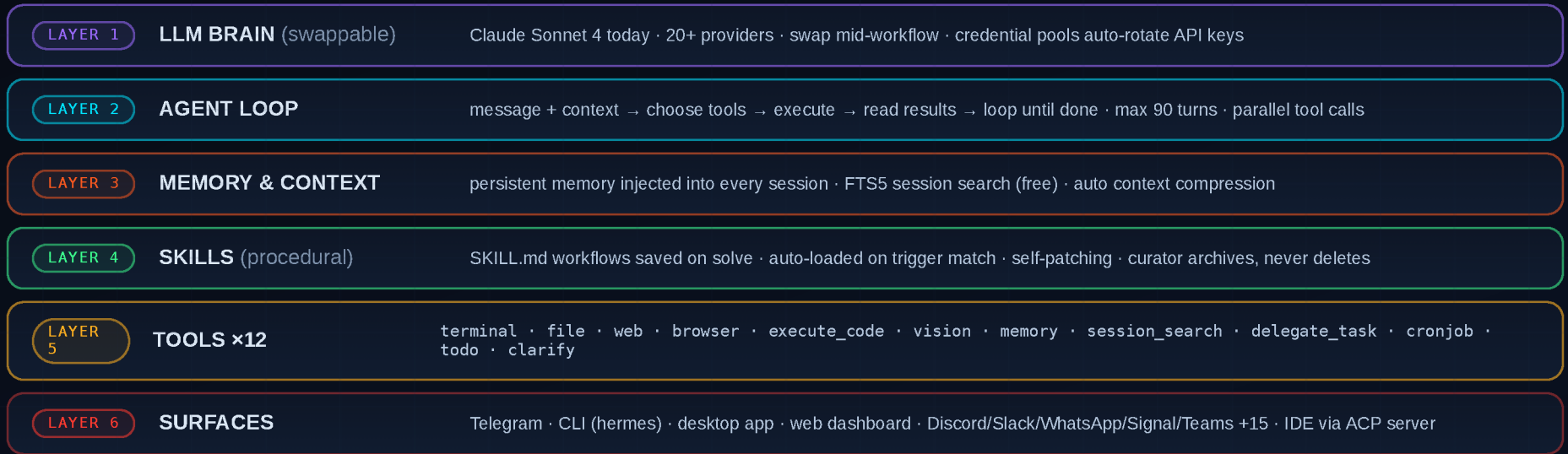
● SIX CAPABILITIES A CHAT WINDOW CAN'T GIVE YOU

- ▶ **Persistent memory** — SQLite-backed sessions, FTS5 full-text search of every past conversation
- ▶ **Full system access** — shell, file ops, browsing, sandboxed code execution, vision
- ▶ **Multi-surface** — Telegram gateway, live-TUI terminal, background sessions in parallel
- ▶ **Autonomous operation** — durable cron jobs & daemonized workflows that outlive the session
- ▶ **Continuous learning** — self-improving skills (e.g. **tree-response-format**)
- ▶ **Deep context** — compression + Obsidian/Claudian integration for long-term pattern recognition

One-line pitch: the model is the brain; Hermes is the nervous system, hands, memory, and calendar.

System Architecture: The Six-Layer Stack

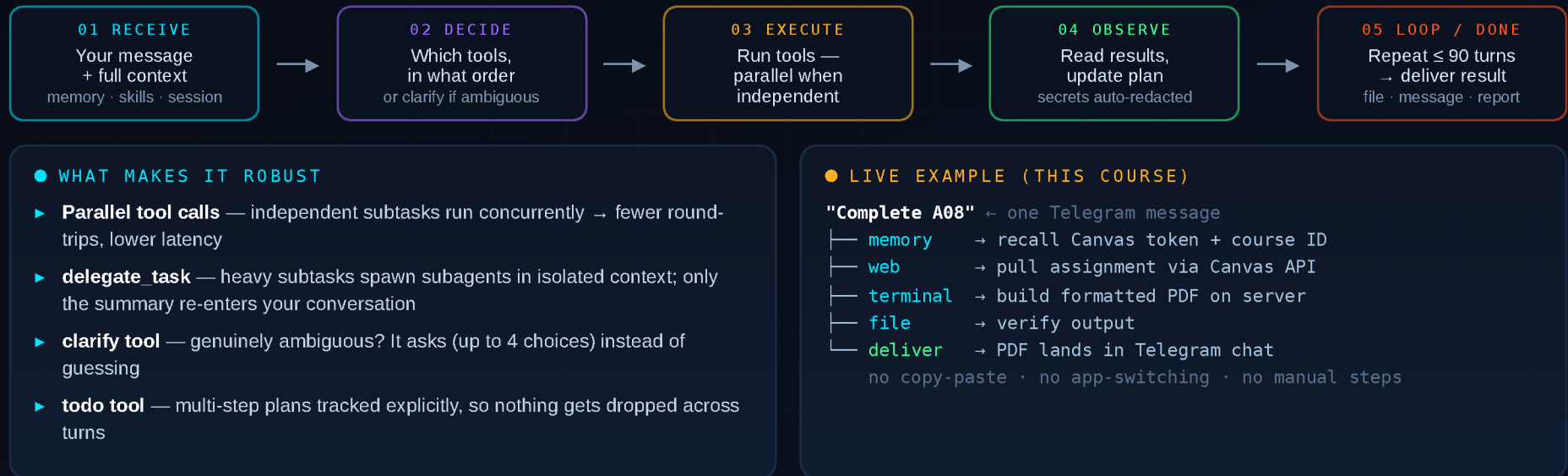
Every request flows down through six composable layers. Each layer is independently configurable — swap the brain, add a surface, or bolt on a tool without touching the rest.



Design principle: intelligence at the top, capability in the middle, presence at the bottom. The same agent that answers you on Telegram executes on the Linux server — one brain, many hands, everywhere you are.

Inside the Agent Loop

The loop is where reasoning becomes action. Hermes iterates — plan, act, observe, refine — up to 90 turns per conversation, batching independent tool calls in parallel, with real-time token & cost monitoring in the TUI.



Memory & Context: Four Layers of Continuity

Not traditional RAG — no vector database, no embedding overhead. Hermes combines curated persistent memory, SQLite session storage with FTS5 full-text search, on-demand skills, and live retrieval. High signal, low token cost.

● PERSISTENT MEMORY

Compact (~1,375-char budget), high-signal facts — name, preferences, Canvas token, class schedule, lessons learned, tool quirks. **Injected into every future conversation automatically.**

● SESSION CONTEXT

Everything said this session lives in the active window. When it grows large, **auto-compression** preserves essentials and drops redundancy — with visual indicators in the TUI.

● SESSION SEARCH (FTS5)

Full-text search across **all past sessions**, SQLite-backed. "What did we work on last week?" → retrieved instantly. Free: no LLM cost, snippets not transcripts.

● LONG-TERM KNOWLEDGE

Obsidian + Claudian integration for pattern recognition and knowledge scoping across weeks of work — plus live **web search & file reading** for anything outside memory.

● TOKEN-FLOW DIAGRAM — WHAT ENTERS THE CONTEXT WINDOW, AND WHEN

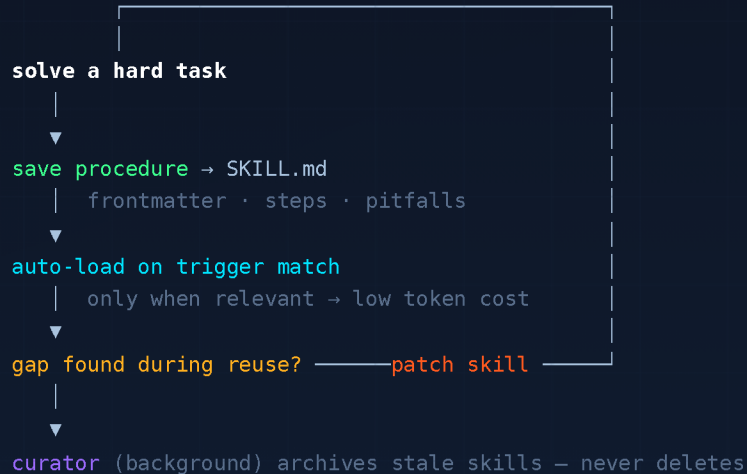
context window (per turn)

- **always** → persistent memory (compact, ~1.4k chars) + current session
- **on trigger** → matching SKILL.md files (loaded only when relevant — token-efficient)
- **on demand** → FTS5 snippets · file reads · web extracts (pulled, not preloaded)
- **offloaded** → `delegate_task` subagents run in isolated context (only summaries return)
- **on pressure** → auto-compression near the limit (essentials preserved, continuity kept)

Skills: Procedural Memory That Compounds

Skills are saved procedures, workflows, and domain knowledge — written as `SKILL.md` files with frontmatter, steps, and known pitfalls. They're created when a complex task is solved, auto-loaded when a task matches their triggers, and patched when gaps are found. This is how the agent gets better every week.

● THE SELF-IMPROVEMENT LOOP



● SKILLS ALREADY SAVED IN THIS DEPLOYMENT

- ▶ `tree-response-format` — the user's preferred structured output style, applied automatically every session
- ▶ `hermes-agent` — how to configure and extend the system itself (the agent documents its own architecture)

● WHY THIS BEATS PROMPTING

- ▶ **Consistency** — the same workflow runs the same way every time, no re-explaining
- ▶ **Token efficiency** — knowledge lives on disk, loaded on demand, not burned in every prompt
- ▶ **Compounding** — every solved problem becomes permanent capability, with persistent feedback across sessions

Key differentiator: chat windows have conversations. Hermes builds a library.

The Full Tool Inventory — 12 Instruments, One Loop

Every tool is callable by the agent loop, independently or in parallel. Color legend: EXECUTION KNOWLEDGE ORCHESTRATION SAFETY

● TERMINAL

Shell commands, scripts, processes — with approval workflow safety controls

● FILE

Read, write, search, patch files — with automatic checkpointing before writes

● EXECUTE_CODE

Sandboxed Python execution with automatic rollback on errors

● BROWSER

Full browser automation — click, fill, navigate — with vision capabilities

● WEB

Live web search and page-content extraction in real time

● VISION

Image analysis and OCR — screenshots, diagrams, documents

● MEMORY

Persistent note-taking and recall across all sessions

● SESSION_SEARCH

FTS5 query of full conversation history — free, no LLM cost

● DELEGATE_TASK

Spawn parallel subagents for complex work in isolated context

● CRONJOB

Schedule autonomous recurring execution — survives session resets

● TODO

Track multi-step task plans so nothing drops across turns

● CLARIFY

Resolve ambiguity before acting — prevents unsafe autonomous execution

● TOOL INTERCONNECTION — HOW THEY COORDINATE INSIDE ONE TURN

```
agent loop → clarify (gate: ambiguous? ask first)
              ↳ todo (plan) → terminal / file / execute_code / browser (act, in parallel)
              ↳ web / vision / memory / session_search (know: feed results back into the plan)
              ↳ delegate_task (offload) → subagent loop → summary returns · cronjob re-enters the loop later, unattended
```


Surfaces: One Agent, Everywhere You Are

The dual-surface architecture pairs a phone-first gateway with server-side execution. Background sessions run daemonized, parallel tasks without conflicts; slash commands give rapid tool-calling in the TUI.

● TELEGRAM — GATEWAY LAYER

- ▶ Message from your phone — **anytime, anywhere**
- ▶ Gateway adapter runs as a persistent background service on the server
- ▶ **Full tool access** — identical capabilities to the CLI
- ▶ Responses return as formatted messages, files, audio, images

● TERMINAL / CLI — LOCAL LAYER

- ▶ Direct shell on the Linux server — scripts, packages, files
- ▶ Live TUI: agent state, token usage, cost, pending decisions
- ▶ Used internally by Hermes for every terminal tool call
- ▶ One-off queries: `hermes chat -q "..."`

● AND BEYOND

- ▶ Native desktop app · web dashboard
- ▶ Discord, Slack, WhatsApp, Signal, Teams + 15 more
- ▶ IDE integration via **ACP server** (VS Code, Zed, JetBrains)
- ▶ Background sessions for daemonized parallel work

● WHY BOTH TOGETHER IS POWERFUL — THE ROUND TRIP

```
phone (Telegram) —"Complete A08"—> gateway adapter —> agent loop on Linux server
                                                                    | Canvas API → PDF build → verify
phone (Telegram) ←—— formatted PDF attachment —————
no copy-paste · no switching apps · no manual steps — trigger remotely, execute natively, deliver back
```

Safety by Design: The Human Stays in Control

Autonomy without control is a liability. Hermes treats human-in-the-loop (HITL) as architecture, not an afterthought — every action is gated, reversible, and traceable.

● COMMAND APPROVAL (CORE HITL)

- ▶ Destructive commands (`rm -rf`, `git reset --hard`, `shred`) require **explicit approval** — requests appear in Telegram before execution
- ▶ Modes: **manual** (default) · **smart** (AI auto-approves safe ops) · **off** ("yolo") — per-session or global via `hermes.yaml`
- ▶ Hardblock / Softblock command lists stop dangerous operations without human review

● CHECKPOINTS & ROLLBACK

- ▶ Automatic snapshots **before** destructive ops — ≥ 1 checkpoint per directory per agent turn
- ▶ Shadow Git repo (`~/.hermes/checkpoints/store/`) — content-addressable, efficient, auto-pruned
- ▶ Restore prior states, undo the last turn, or preview first: `/rollback diff N`

● SECURITY LAYERS

- ▶ **Secret redaction** — keys/tokens scrubbed from tool output before entering context; **cannot be disabled mid-session by the model itself** (deliberate)
- ▶ SSRF protection · homograph attack detection · supply-chain verification of dependencies
- ▶ Container isolation (Docker/Singularity) · pairing-code user authorization · MCP for secure credential handling · audit logging for compliance

● BOUNDED AUTONOMY

- ▶ **No action between your messages** unless you explicitly scheduled a cron job
- ▶ **Clarify before acting** — ambiguity triggers a question, not a guess
- ▶ **Threat pattern scanner** — project context files (AGENTS.md, .hermes.md) are scanned; prompt-injection attempts are blocked, not silently passed
- ▶ Granular interrupts: pause mid-task, review, resume or redirect — every action traceable in the session log

The safety thesis: a trustworthy agent isn't one that never acts — it's one whose every action is **gated by policy, reversible by checkpoint, and visible in the log.**

Isolation: Git Worktrees & Multi-Agent Profiles

Parallel agents are only useful if they can't step on each other. Hermes isolates at two levels — the codebase (worktrees) and the agent identity (profiles).

● GIT WORKTREES — PARALLEL EXECUTION

```
main repo (untouched)
├─ agent A → own branch · clean working dir
│   · isolated Checkpoint Manager
├─ agent B → experimental refactor, same task,
│   different approach — zero conflicts
└─ hermes -w → disposable worktree, auto-created
```

- ▶ No interference with the main codebase
- ▶ Clean, reviewable pull requests · easy error recovery

● PROFILES — INDEPENDENT AGENTS

~/.hermes-coder/	~/.hermes-researcher/
├─ config.yaml	├─ config.yaml
├─ .env (own keys)	├─ .env (own keys)
├─ SOUL.md (personality)	├─ SOUL.md
├─ memory · sessions	├─ memory · sessions
└─ skills	└─ skills

- ▶ Complete isolation — no API-key conflicts, no memory bleed
- ▶ Per-profile gateway supervision · crash recovery via supervised sub-processes
- ▶ Clone, export, import, share profiles from Git repositories

Result: you can run a specialized coder, a researcher, and a scheduler simultaneously — different personalities, different credentials, different memories — on one machine, with containerized deployment for reproducibility and portability.

How Hermes Answers Today's Agentic Engineering Concerns

Each known failure mode of autonomous agents maps to a specific, implemented mechanism — not a roadmap item.

CONCERN	HERMES MECHANISM
Unsafe autonomous execution	Checkpoints + rollback · approval workflows · Hardblock/Softblock lists · container isolation · clarify-before-acting
Credential leakage	Automatic secret redaction (un-disableable by the model) · secure env pass-through · MCP credential management
Session loss	Persistent SQLite session storage · background sessions · cron jobs survive resets
Token overhead	Compact memory (~1.4k chars) · on-demand skills · context compression · free FTS5 search · subagent offloading · token-flow tracking
Agent conflicts	Git worktrees per agent · profile isolation · supervised sub-process crash recovery
Context fragmentation	Persistent memory injection · Obsidian + Claudian long-term knowledge scoping · session search continuity
Context window limits	Real-time window monitoring · compression indicators in TUI · auto-compression near limit
Interrupt management	Granular pause / review / resume / redirect via TUI · task queues · manual override & escalation paths
Insufficient human oversight	Command approval · clarify tool · audit logging · real-time monitoring dashboard · per-session policies in hermes.yaml

Hermes vs. Standalone GPT / Claude

Same underlying intelligence — radically different capability envelope. This is the difference architecture makes.

CAPABILITY	STANDALONE CHAT (GPT / CLAUDE.AI)	HERMES AGENT
Memory across sessions	x stateless – forgets everything	✓ persistent memory + FTS5 search of all history
System access	x none	✓ terminal · files · browser · APIs · sandboxed code · vision
Autonomous / scheduled work	x dies with the tab	✓ durable cron jobs · daemonized background sessions
Learning & improvement	x same experience every time	✓ self-improving skills, patched & curated over time
Interaction surfaces	x one web chat	✓ Telegram · CLI/TUI · desktop · IDE · 15+ platforms
Parallelism	x one thread	✓ parallel tool calls · subagents · worktrees · profiles
Safety & reversibility	– n/a (can't act at all)	✓ approvals · checkpoints · rollback · redaction · audit logs
Brain flexibility	x locked to one vendor	✓ swappable across 20+ providers, mid-workflow
Deliverables	x text you copy out	✓ real files: PDFs, scripts, reports – sent to your phone

The honest caveat: the chat window is simpler, and for one-off questions it's enough. Hermes earns its complexity the moment a task must **persist, execute, repeat, or improve** — which is most real work.

This Isn't a Demo — It's Running Right Now

Everything below is the actual configuration and confirmed capability set of a live deployment built for this course.

● ACTIVE CONFIGURATION

deployment

```
|— model    → Claude Sonnet 4 (Anthropic)
|— surface  → Telegram (@InVictuSmurff_bot)
|— platform → Linux server · Python 3.13
|— memory (persisted)
|   |— Canvas token + HCCS Eagle Online URL
|   |— ITAI-1370 course ID · user ID
|   |— Teams links (Jul 14 · 16 · 21 · 23)
|   |— response format preference (trees)
|— skills (active)
|   |— tree-response-format
|   |— hermes-agent
```

● CAPABILITIES CONFIRMED THIS SESSION

- ▶ **Canvas API** — read assignments, grades, submission status
- ▶ **PDF generation** — formatted academic documents from multi-step workflows
- ▶ **File delivery** — documents sent directly to Telegram
- ▶ **Memory** — credentials & preferences persist across sessions
- ▶ **Skill creation** — workflows saved and reused automatically

● CRON: AUTONOMY IN PRACTICE

- ▶ **every sunday 8pm** → check Canvas for new assignments
- ▶ **every friday** → summarize what's due next week
- ▶ **1h before class** → send the Teams meeting link
- ▶ Jobs run in fresh sessions, survive resets, deliver to Telegram, a file, or any connected channel

End-to-end pattern: long-running workflows with persistent memory · code execution with auto-rollback · parallel background tasks without conflicts · custom personality via **SOUL.md** · human checkpoints wherever they matter.

Why This Matters

The frontier of AI is no longer just model quality — it's **agentic engineering**: the discipline of giving models memory, hands, judgment gates, and accountability. Hermes demonstrates that responsible, controllable autonomy is buildable today, by a student, on a Linux server.

● FOR ENGINEERING

Parallel agents on isolated worktrees, checkpointed and reviewable — AI that ships PRs without endangering `main`.

● FOR RESEARCH & STUDY

A memory-bearing collaborator that tracks your courses, retrieves last week's reasoning, and does the retrieval for free.

● FOR AUTONOMOUS SYSTEMS

A reference pattern for bounded autonomy: act only within policy, snapshot before risk, log everything, ask when unsure.

\$ the takeaway

Intelligence is rented from the model. Capability is built in the architecture.

explore & build your own → hermes-agent.nousresearch.com/docs/user-stories ■